

UCS503P - Software Engineering Project

ChargEV: Find. Charge. Go Green.

The Smart EV Charging Station Platform

Submitted by:

1024170103 Pehal
1024170431 Briti Arora

Group: BE Third Year, CSE

Submitted to:

Dr. Nisha Thakur

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	3
1.1	Background and Context	3
1.1.1	Problem Statement	3
1.2	Project Objectives	3
2	Methodology and Design	5
2.1	System Architecture	5
2.1.1	High-Level Design and Modelling	5
2.1.2	Technical Stack	5
2.2	Implementation Details	6
2.2.1	Module 1: Frontend UI/UX and Prototype	6
2.2.2	Module 2: Backend Development Setup	6
2.3	System Modeling and Project Diagrams	9
3	Results and Evaluation	15
3.1	Testing Methodology	15
3.2	Performance Metrics	15
3.3	Analysis	16
4	Conclusion	17
4.1	Summary of Work	17
4.2	Key Findings	17
4.3	Future Work and Recommendations	17
4.4	Final Remarks	18

Introduction

1.1 Background and Context

Electric vehicle (EV) adoption is accelerating, but the charging infrastructure that supports it has not kept pace in terms of usability. Drivers routinely do not know which nearby stations are actually free, whether a connector will be compatible with their vehicle, or whether they will have to wait once they arrive. This project, **ChargeEV**, aims to remove this uncertainty by giving EV drivers a single, reliable platform to discover charging stations, check real-time availability, and reserve a slot in advance – turning charging from a source of anxiety into a predictable, routine task.

1.1.1 Problem Statement

While EV charging infrastructure continues to expand, the tools available to EV drivers to actually use that infrastructure remain fragmented and unreliable. The specific problems this project addresses are:

- **Discovery gap:** it is hard to find nearby charging stations from a single, reliable source.
- **Unclear availability:** station status (free, occupied, out of service) is rarely known before arriving.
- **Wasted time:** drivers wait at busy stations with no way to reserve a slot in advance.
- **Generic search results:** existing tools do not let drivers filter by connector type, distance, availability, and price together.
- **Difficult trip planning:** long-distance EV travel requires manually chaining together charging stops.
- **Low engagement:** there is little incentive for drivers to keep returning to a single platform.

As EV ownership grows, even small improvements in charging predictability translate into measurable reductions in range anxiety and wasted driver time.

1.2 Project Objectives

1. **Centralized Station Locator:** Build a single source of truth for nearby EV charging stations, searchable by location.
2. **Real-Time Availability & Smart Booking:** Deliver a live availability dashboard and an online slot booking and reservation system so drivers can guarantee a charge before they leave.
3. **Smart Search and Trip Planning:** Provide filters by connector type, distance, availability and price, and extend this into route-aware trip planning for long-distance travel.

4. **Engagement Layer:** Implement a gamified rewards system (“EcoCoins”) that tracks savings and eco-impact to encourage repeat usage.

Methodology and Design

2.1 System Architecture

2.1.1 High-Level Design and Modelling

Before writing any implementation code, the team invested the first four weeks of the semester in requirement analysis and system modelling, so that the frontend and backend work started in parallel could stay consistent with a shared blueprint. This involved:

- A **UML Use Case Diagram** identifying the system boundary and its actors – the EV Driver (primary actor), the Station Operator (secondary actor), and the Location Services API and MongoDB Atlas as supporting actors – along with the main features: searching stations, applying smart filters, checking real-time availability, booking a slot, navigating to the station, planning multi-stop trips, and earning EcoCoins.
- An **Activity Diagram** mapping the step-by-step workflow of a station search, from entering a location and filters to displaying results on the map.
- A **Swimlane Activity Diagram** that distributes the complete search-book-charge flow across four responsibilities: the EV Driver/Frontend, the Backend API, Location Services (Maps/Places/ Directions), and MongoDB Atlas – covering everything from the initial search through booking, navigation, charging, and the final eco-impact summary.
- A **Sequence Diagram** describing the chronological, message-level interaction between the User, the Frontend, the Backend, the Database (MongoDB Atlas), and the Google APIs during a station search and booking.
- A **Collaboration Diagram** representing the same set of interactions as a structural map of communication links between those same components, with each link numbered to show the order of messages.
- A **Gantt Chart** laying out the full 20-week development plan, from requirement analysis through to the final report and demo.

All diagrams were cross-checked against the project proposal to ensure consistency between the stated requirements, the workflow, and the proposed system architecture, and are presented in full in Section 2.3.

2.1.2 Technical Stack

- **Frontend:** HTML5, CSS3, Bootstrap 5, and JavaScript, built for speed and cross-device responsiveness.
- **Backend:** Node.js and Express.js, chosen for high-performance, event-driven request routing.
- **Database:** MongoDB Atlas, a cloud-native document database.
- **Location Intelligence:** Google Maps API, Google Places API, and Google Directions API, used for station search, geocoding, and route planning.

2.2 Implementation Details

2.2.1 Module 1: Frontend UI/UX and Prototype

Frontend work began with research into interface patterns used by other map-based and booking platforms, which shaped the decision to use a map-plus-station-list layout with a simplified booking flow of *station selection* → *slot selection* → *confirmation*. Based on this research, a fully navigable multi-page interface was implemented using HTML5, CSS3 and Bootstrap 5, styled around a verified, dark-themed EV charging brand identity:

- A **Home page** with a hero search bar, quick filters (connector type, availability, 24/7 open), and a live-style preview card for the nearest verified station.
- A dedicated **Stations page** combining a scrollable station directory with an interactive map view, so a driver can compare stations by price, connector type, and timings before choosing one.
- A **My Bookings** page listing a driver’s scheduled charging sessions – booking ID, station, connector, time window, duration, cost, and status – in a clean, tabular layout.
- A **Dashboard** presenting an active charging session with live-style telemetry (state of charge, charging rate, energy delivered, elapsed time, and accrued cost), alongside a saved-bookings table, in the format the platform will use once connected to the live backend.
- A **Create Account** flow for onboarding new drivers onto the Patiala EV charging network.

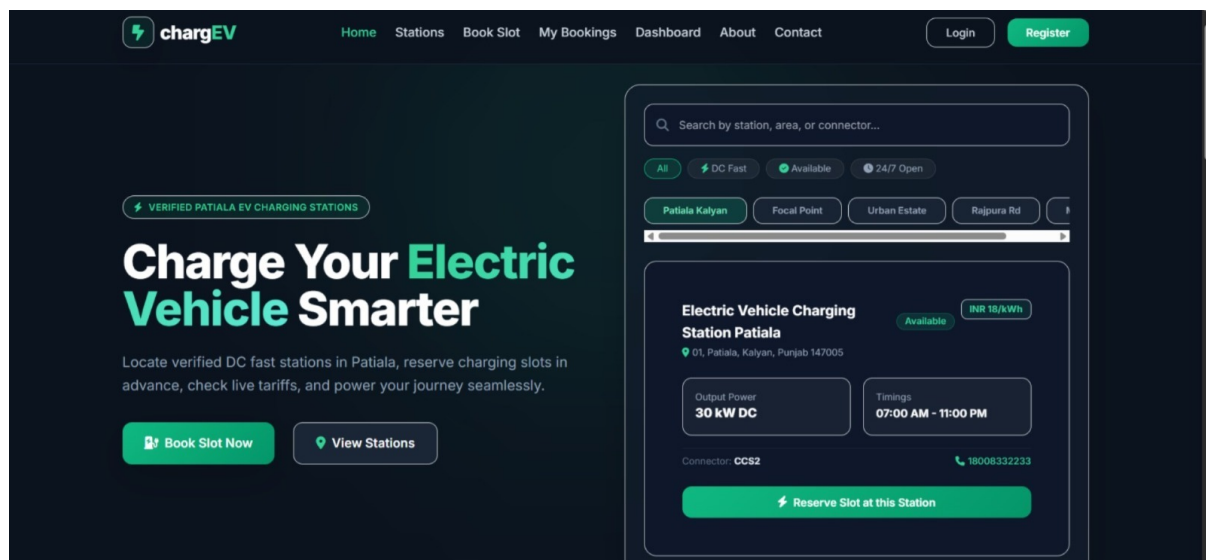


Figure 2.1: ChargeEV Home Page: hero search, quick filters, and nearest-station preview.

The layout was made **responsive** across desktop and mobile screens using Bootstrap’s grid system, and the full navigation flow – Home, Stations, Book Slot, My Bookings, and Dashboard – was validated end to end using mock and simulated data, confirming that every screen correctly renders the data structures the backend is being built to supply.

2.2.2 Module 2: Backend Development Setup

In parallel, the initial backend environment was set up so that frontend-backend integration can begin as soon as the first API endpoints are ready:

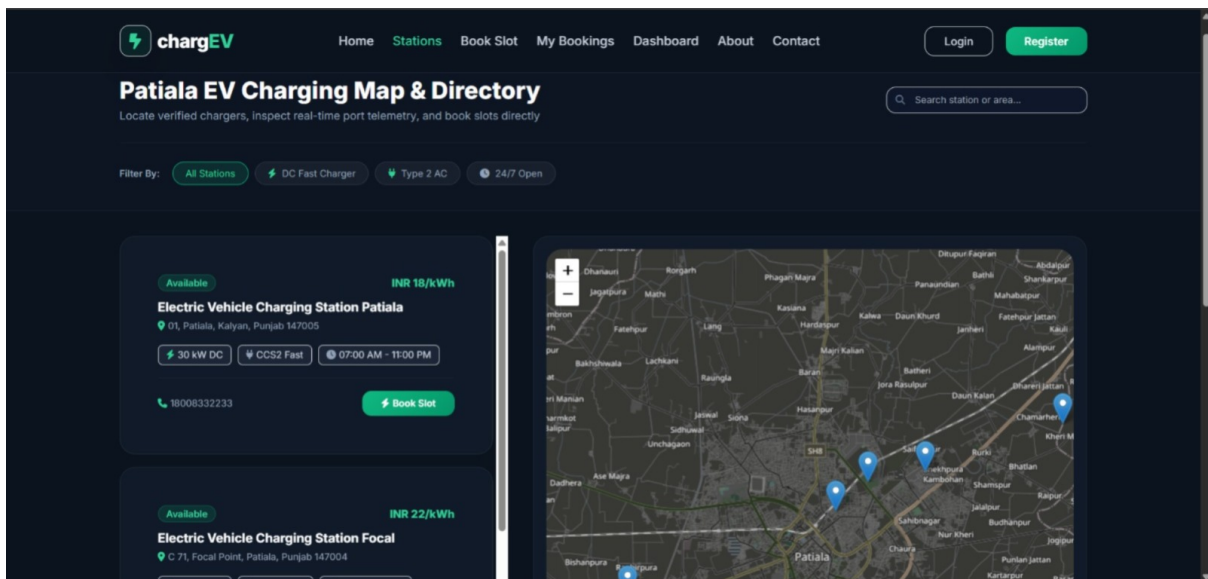


Figure 2.2: Stations Page: Patiala EV charging directory with combined list and map view.

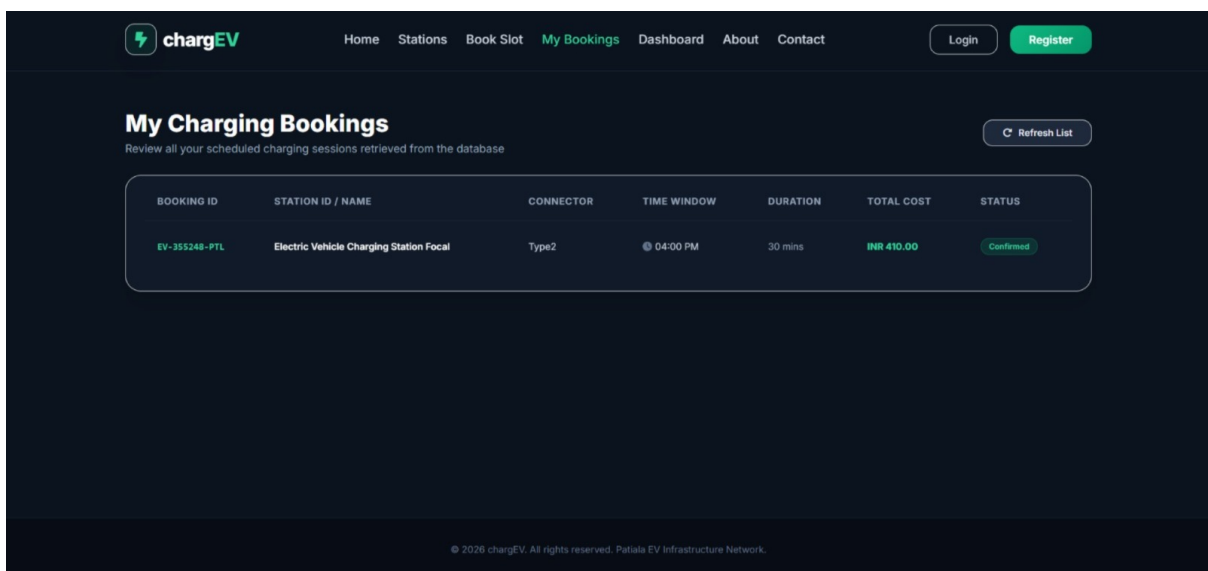


Figure 2.3: My Bookings: a driver's scheduled charging sessions in a clear tabular view.

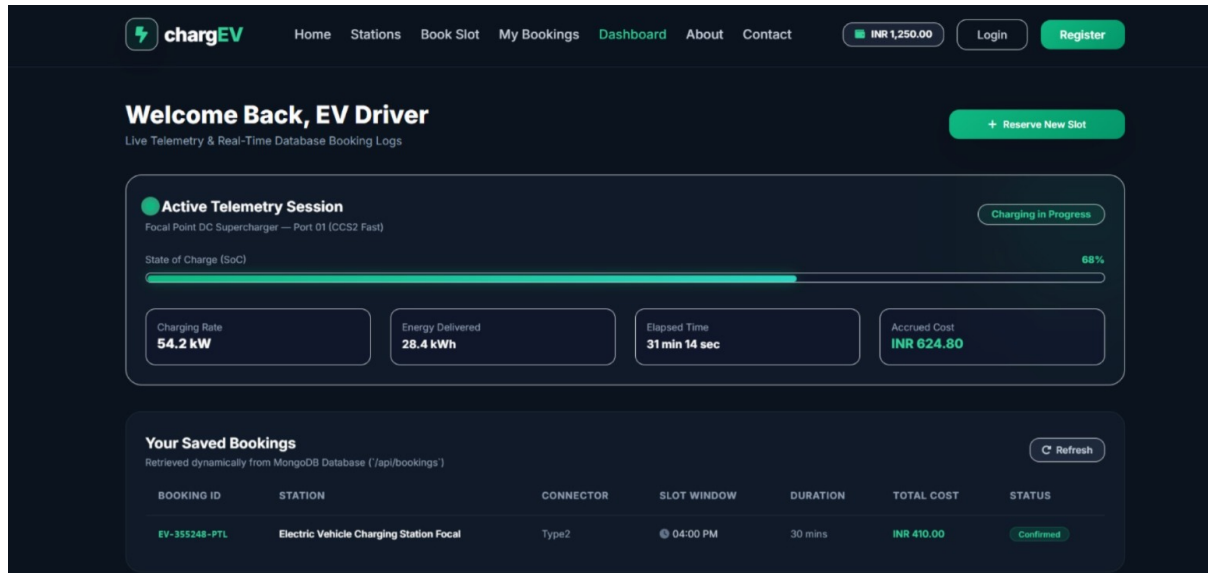


Figure 2.4: Dashboard: live-style charging telemetry alongside saved booking records.

The 'Create Account' form is centered on the page. It includes fields for 'FULL NAME', 'EMAIL ADDRESS', and 'PASSWORD'. A 'Create Account' button is at the bottom, with a link to 'Sign In' for existing users.

Create Account
Join the Patada EV charging grid and network.

FULL NAME
Buntd

EMAIL ADDRESS
nabin@example.com

PASSWORD
Create a secure password

Create Account

Already have an account? Sign In

Figure 2.5: Create Account: onboarding flow for new EV drivers.

- The **Node.js** and **Express.js** server environment was initialised.
- The connection path between the **frontend, backend, and MongoDB Atlas** was explored and documented.
- The core backend modules were identified: **users, stations, and bookings**.
- Initial planning was done for the REST API structure that will expose these modules to the frontend in the next iteration.

At this stage, the backend is a scaffolded environment rather than a working API – no endpoints, database schema, or live Google API calls have been implemented yet. This is the next planned milestone (see Section 4.3).

2.3 System Modeling and Project Diagrams

To provide a comprehensive blueprint of the ChargeEV system, the following diagrams were developed and finalized during Weeks 2–4 of the semester.

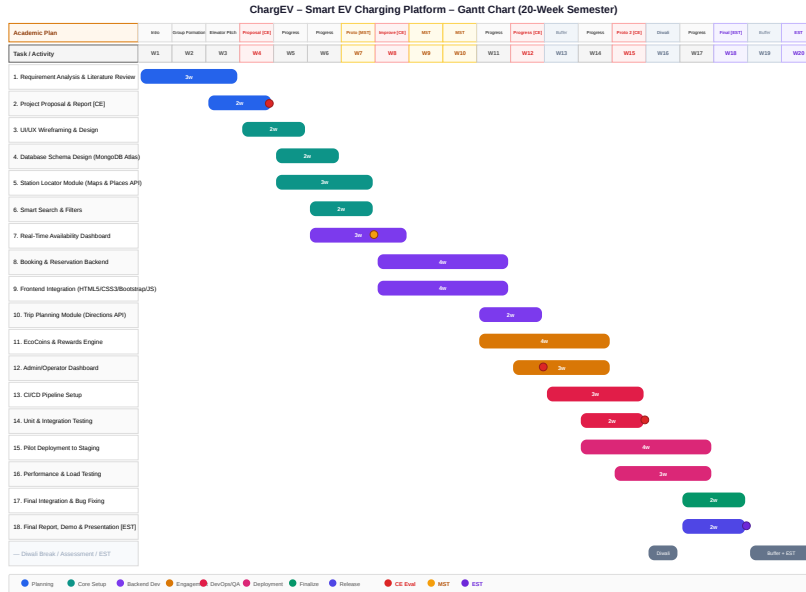


Figure 2.6: ChargeEV project timeline (Gantt Chart) across the 20-week semester.

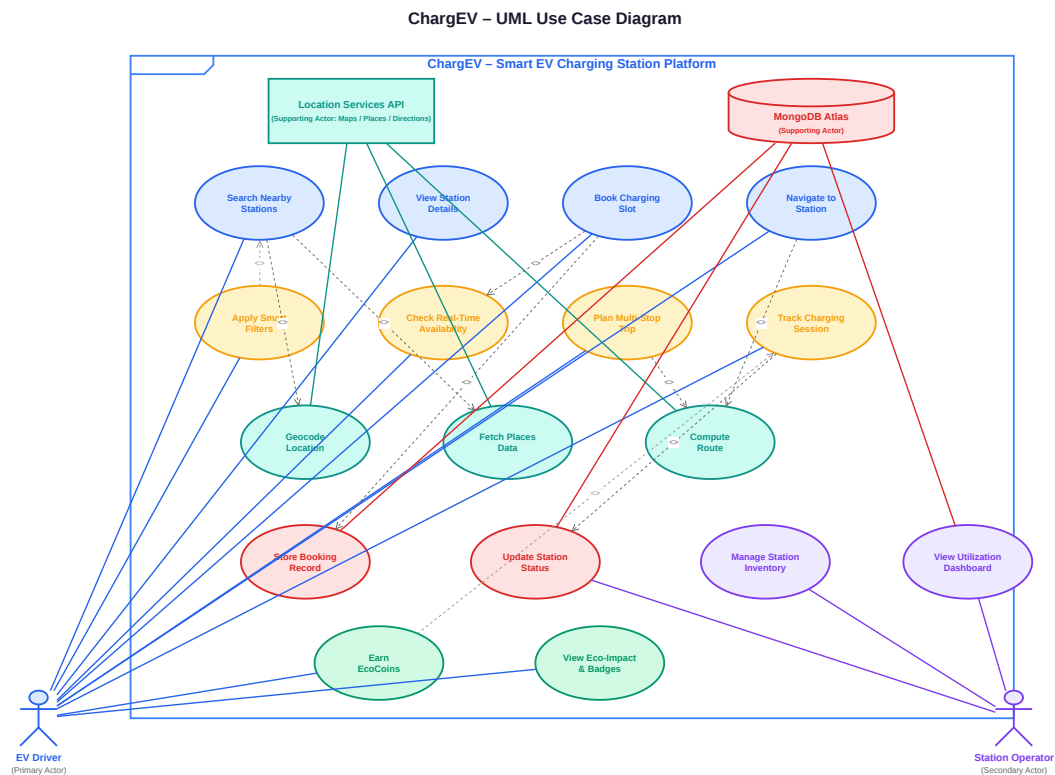


Figure 2.7: UML Use Case Diagram identifying the EV Driver, Station Operator, and supporting actors.

ChargEV – Activity Diagram (Station Search)

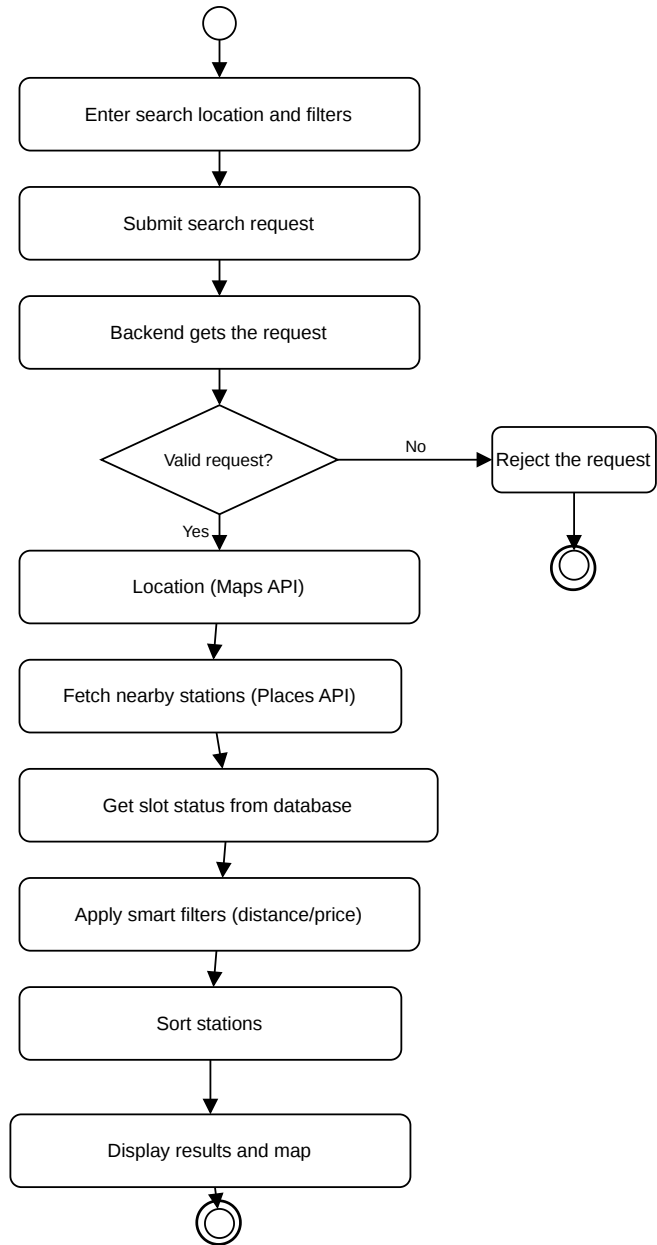


Figure 2.8: UML Activity Diagram for the station search workflow.

ChargeEV: Complete Search-Book-Charge Flow

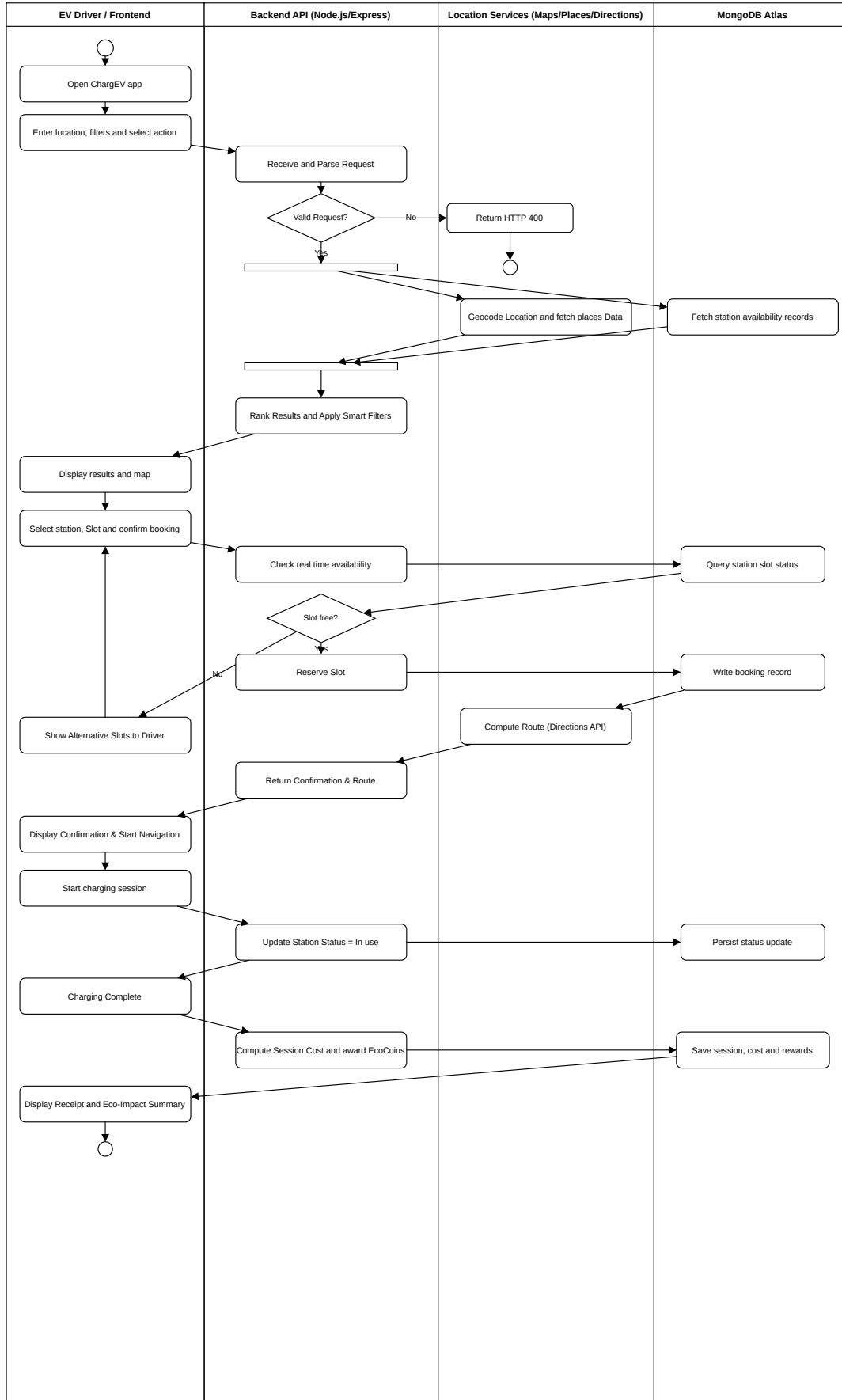


Figure 2.9: UML Activity Diagram mapped across swimlanes: EV Driver/Frontend, Backend

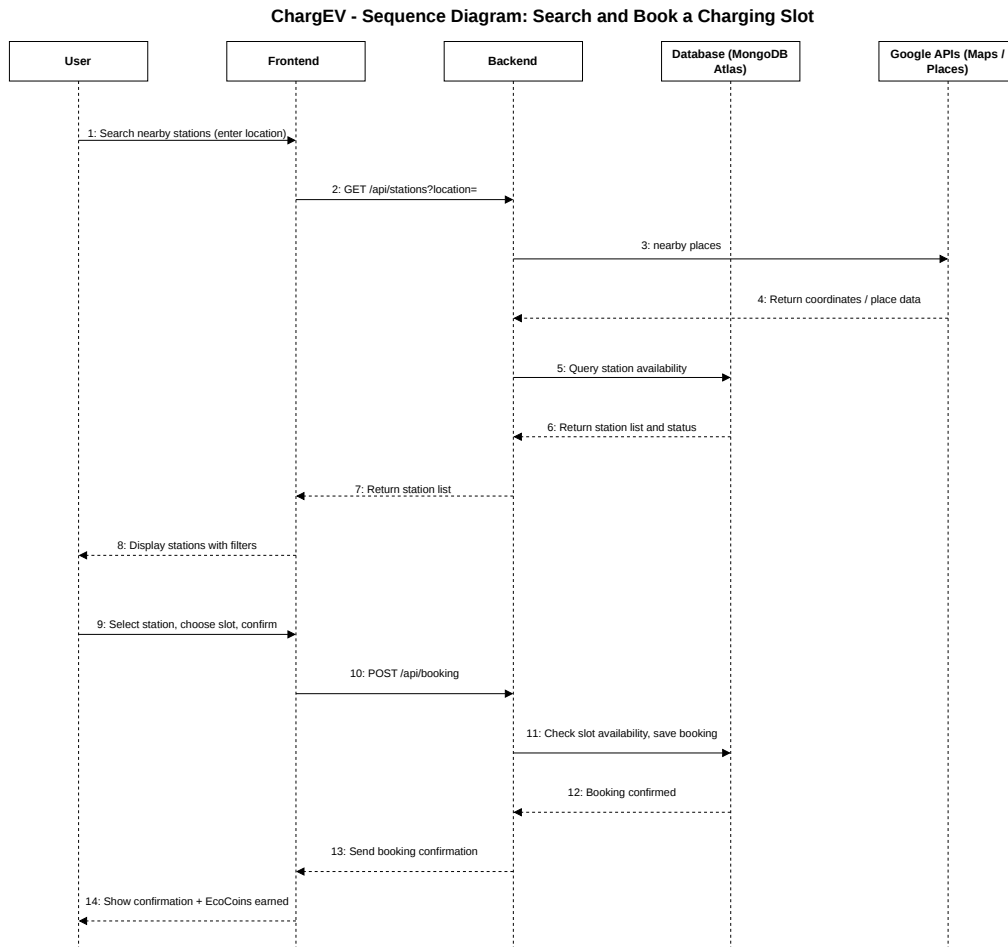


Figure 2.10: UML Sequence Diagram depicting the chronological interaction between system components during a station search and booking.

ChargeEV - Collaboration Diagram: Search and book a charging slot

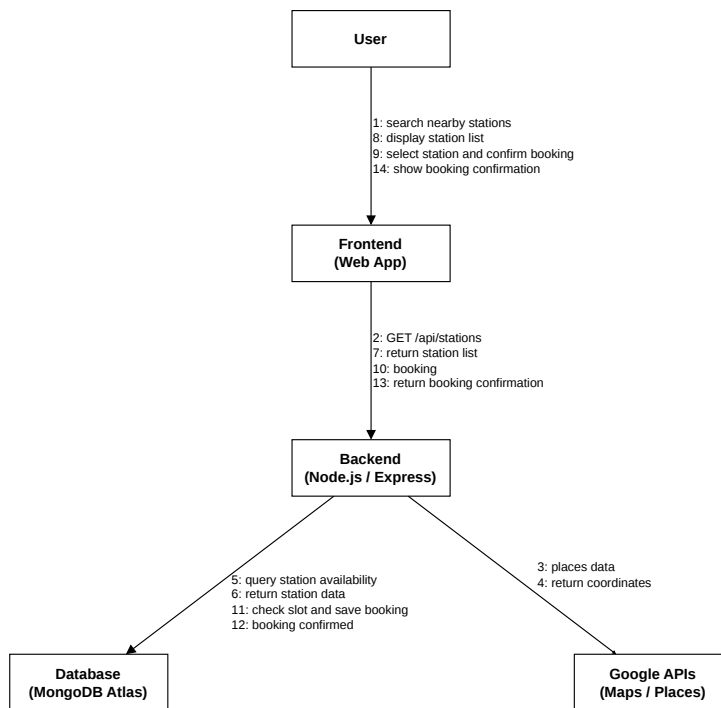


Figure 2.11: UML Collaboration Diagram highlighting the structural communication links between components.

Results and Evaluation

3.1 Testing Methodology

As this is the mid-semester evaluation stage, our efforts have successfully culminated in a validated system blueprint and a fully navigable frontend prototype for ChargeEV:

- **UML Design Validation:** The comprehensive UML models – Use Case, Activity, Swim-lane Activity, Sequence, and Collaboration diagrams – have been cross-checked against the project proposal and validated as the logical blueprint for the discover-book-charge workflow.
- **UI/UX Validation:** The frontend has undergone rigorous interface validation across five core screens – Home, Stations, My Bookings, Dashboard, and Create Account. The layout maintains responsiveness across desktop and mobile viewports, and input integrity is enforced through HTML5 attributes and JavaScript event listeners.
- **Mock Integration & Flow:** The interface transitions seamlessly from station discovery on the Home and Stations pages, through booking, to the Dashboard and My Bookings views. The mock and simulated data used at this stage successfully populates every station card, booking row, and telemetry widget, confirming that the frontend is fully capable of rendering the data structures the backend is being built to supply.
- **Backend Environment Validation:** The Node.js and Express.js server environment was initialised and validated as ready to host the `users`, `stations`, and `bookings` modules, with the MongoDB Atlas connection path confirmed ahead of full API development.

3.2 Performance Metrics

To keep development outcome-driven, the proposal defines the metrics framework that will validate ChargeEV's booking workflow as backend integration proceeds:

Metric	Definition / Target
Time-to-Charge (TTC)	Time from opening the app to starting a charge at a booked slot; target is a reduced median TTC versus a no-booking, walk-in baseline.
Booking Conversion Rate	Percentage of station searches that convert into a confirmed slot booking.
Availability Data Accuracy	Agreement between the displayed station status and the actual status observed on arrival.
User Engagement	Repeat-usage rate and EcoCoins accumulated per active user.

Table 3.1: Evaluation metrics framework for the ChargeEV booking workflow.

This framework ensures that once the backend and live station data are connected, ChargeEV's success can be measured objectively against the same discover-book-charge workflow already validated in the UI and system diagrams.

3.3 Analysis

Our progress strictly adheres to our initial Gantt chart objectives for the Requirements, UI/UX, and Backend Setup phases shown in Figure 2.6. The robust, multi-page, mock-data-driven design of the frontend ensures that integrating the REST API will require minimal UI refactoring, since every screen has already been validated against the same data shapes the backend modules are designed to return. The parallel system modelling effort – particularly the Swimlane Activity Diagram and the Sequence and Collaboration diagrams – gave both halves of the team a shared contract to build against, so that the frontend and backend work completed independently in the first four weeks will integrate cleanly. Moving forward, the primary engineering focus will shift to implementing the `stations` and `bookings` REST endpoints and connecting them to the already-validated frontend and location-intelligence architecture.

Conclusion

4.1 Summary of Work

To date, the team has:

- Defined the problem scope and finalized the project proposal and technical stack.
- Completed a full set of UML and project diagrams – Gantt Chart, Use Case, Activity, Swimlane Activity, Sequence, and Collaboration – validated against the proposal.
- Built and validated a fully navigable, responsive frontend prototype spanning the Home, Stations, My Bookings, Dashboard, and Create Account screens.
- Initialised and validated the Node.js/Express backend environment and its planned connection to MongoDB Atlas.

4.2 Key Findings

- **Finding 1:** Building the frontend first, in the shape defined by the Use Case and Sequence diagrams, means the planned REST API can be dropped in with minimal UI refactoring later.
- **Finding 2:** The Swimlane Activity Diagram clarified responsibility boundaries between the Backend API and Location Services early, before any backend code was written, which directly shaped how the `stations` and `bookings` modules were scoped.
- **Finding 3:** Designing the Dashboard and My Bookings screens around the exact fields the MongoDB schema will store (booking ID, connector, time window, cost, status) gives the team a validated target schema to implement against in the next iteration.

4.3 Future Work and Recommendations

Following the established Gantt chart timeline, the immediate next steps are:

1. **REST API Development:** Implement the `stations`, `bookings`, and `users` endpoints in Express and connect them to a MongoDB Atlas schema.
2. **Location Intelligence Integration:** Integrate the Google Maps, Places, and Directions APIs for live station search, geocoding, and routing.
3. **Frontend-Backend Integration:** Connect the validated frontend screens to live API responses, so the already-built UI is driven end-to-end.
4. **Trip Planning and EcoCoins:** Build the route-aware trip planning module and the EcoCoins rewards and eco-impact tracking engine.
5. **Testing and CI/CD:** Add unit and integration tests, and set up a CI/CD pipeline for staged, incremental releases.

4.4 Final Remarks

The first four weeks of this project have established a consistent, validated blueprint – across the proposal, the full UML diagram set, and a fully navigable frontend prototype – for the discover-book-charge workflow that ChargeEV is built around. This foundation is intended to make the coming integration phase, where the frontend, backend, and external APIs are connected end-to-end, significantly more predictable.